Recap Checkpoint 1 — after 1.1 (Processors, Input/Output & Storage) + 1.2 (Software) — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Mark scheme

Total = 34 marks. Award marks for valid alternatives in line with OCR positive-marking.

Q1 — [2] (AO1) — *1.1.1* - PC holds the **address of the next instruction** to be fetched (1). - CIR holds the **current instruction** being decoded/executed (1).

Q2 — [3] (AO1) — *1.1.2* (a) Pipelining **fetches, decodes and executes different instructions at the same time** (1) so that while one instruction is executing, the next is decoded and a third is fetched — increasing throughput/instructions completed per unit time (1). (b) Any one (1): a **branch/jump** (conditional jump) means the pre-fetched instructions are wrong / a branch misprediction / an interrupt occurs. *Examiner tip: pipelining improves throughput, not the time for a single instruction.*

Q3 — [3] (AO2) — *1.1.2* - A GPU has **many (hundreds/thousands of) cores** (1)… - …suited to **SIMD / massively parallel** processing of the same operation on many data items (1)… - …so each pixel's identical calculation runs in parallel, whereas a CPU has few cores optimised for sequential/varied tasks, making it slower for this workload (1).

Q4 — [3] — *1.1.3* (a) **Flash (solid-state)** storage (1). *(AO1)* (b) Any two (1 + 1) *(AO2)*: **no moving parts** so it is more shock/drop resistant in a portable device; **lower power consumption** (battery life); **smaller/lighter/more compact**; **faster access** times; silent. *(Reject "more storage" — HDDs offer more capacity per £.)*

Q5 — [4] (AO1) — *1.2.1* Award up to 4 from: - The CPU **checks for interrupts at the end of each FDE cycle** (1). - The current **contents of registers / state are saved onto the stack** (1). - The appropriate **ISR is loaded/run** to service the interrupt (1). - Interrupts are **prioritised** — only a higher-priority interrupt suspends the current ISR (1). - When the ISR finishes, the **saved state is restored (popped) from the stack** and the original program resumes (1).

Q6 — [3] — *1.2.1* (a) Virtual memory uses a **portion of secondary storage as if it were RAM** (1), so programs **larger than physical RAM can run** / more programs can be held than fit in RAM (1). *(AO1)* (b) Heavy use causes **disk thrashing** — constant paging in/out of pages slows the system because secondary storage is far slower than RAM (1). *(AO2)*

Q7 — [4] (AO1) — *1.2.2* (a) Any one difference for 2 marks (1 for each side, or 1+1 for a developed point): - A compiler **translates the whole program at once** producing an executable; an interpreter **translates and executes line by line** (1). - Compiled code **runs faster / no translator needed at run time**; interpreted code is **easier to debug / portable** (1). (b) Correct order (2 marks; 1 mark if one pair transposed): **lexical analysis → syntax analysis → code generation → optimisation**. *Examiner tip: lexical (tokenise) before syntax (parse); optimisation refines the generated code.*

Q8 — [4] (AO2) — *1.2.3* - Recommend **Agile / extreme programming** (1)… - …because requirements are **expected to change**, and Agile is **iterative**, delivering working increments and welcoming change late in development (1). - Second characteristic (1): **frequent customer/stakeholder involvement** ensures the product matches evolving needs / **frequent small releases**, **pair programming**, or **continuous testing** improves quality. - Justified link back to the requirement of changing needs (1). *(Accept a justified Waterfall answer only if it engages with — and overcomes — the "changing requirements" point; otherwise no credit for choosing Waterfall here.)*

Q9 — [3] — *1.2.4* (a) Encapsulation **bundles data (attributes) and the methods that act on them within an object** (1) and **restricts direct access to the data (private attributes accessed via methods)** — information hiding (1). *(AO1)* (b) Inheritance lets a subclass **reuse code from a superclass** (1) — less duplication / easier maintenance / models an "is-a" relationship. *(AO2)*

Q10 — [5] (AO3) — *synoptic 1.1 + 1.2* Mark holistically; up to 5 from a reasoned evaluation covering **both** decisions: - **Processor:** recommend **RISC** (1) — simpler, fixed-length instructions, lower power and predictable execution suit long reliable shifts / embedded-style tills, with complexity handled in software; *(or a justified CISC choice citing fewer instructions per task)* (1 for justification). - **Operating system:** recommend a **multi-tasking** OS (1) so staff can run **card payments and stock look-ups concurrently** via time-slicing; a **real-time** OS could be argued for guaranteed response, distributed rejected as a single till (1 for justified link). - **Evaluation/trade-off** that weighs the choices against reliability, fast recovery and concurrency (1). *Indicative full-mark answer: RISC for low-power predictable operation + multi-tasking OS for concurrent transactions, with a trade-off noting RTOS could guarantee responsiveness but adds complexity.*